Adaptive Image Reads for Agent Training

Majd Zayyad

November 2026

Abstract

An agent-training job can leave its learner waiting for tools that are still fetching files from a remote image. Reading larger chunks reduces requests but can waste bandwidth; smaller chunks have the opposite cost. We study a reader that chooses how much to fetch from recent access patterns and observed request cost. The question is whether this local decision shortens training execution outside model inference. We combine measured image-read jobs with a scheduler that counts setup, tool computation, queues, verification, teardown, and learner updates, while excluding inference service and its queue. Across the calibrated read conditions, inference-excluded execution time is 25.7% lower when the other stages are short, and 9.6% lower with long learner updates, relative to fixed256. The gains depend on read locality and the remaining work; losing stream identity can reverse them. The experiments explain when image-read savings reach the learner, and when other work absorbs them.

1 Why image reads matter

Agentic training repeatedly turns model output into actions: inspect a repository, import a library, compile a program, or run tests. Those tools need files inside their sandbox. With remote-backed images, some of the files arrive only when read. If tools finish late, a training update can wait even when the learner itself is ready.

DSec makes the workload mismatch concrete. It reuses 3FS, its training-storage backend, for on-demand sandbox images. Section 5.3 reports strong throughput for large sequential I/O but poor performance for small random requests [3]. DSec bridges that gap with local writes, local metadata where possible, and bulk reads into caches. Its OverlayBD/ublk path fetches 256 KiB chunks. Agent tools, however, often want a few files promptly, then move to another short burst of work.

This follows the familiar *I/O-granularity trade-off*. Large fetches amortize request overhead; small ones limit read amplification—bytes transferred but not used. Lambda's image-loading study describes this same random-access versus sequential-throughput trade-off [1]. The opportunity is to adapt the client-side fetch size to the current access pattern, rather than require a new backend or a larger cache.

Read-ahead already adapts to sequential access, as in AMP [2]. TIP and Khameleon connect prefetching decisions to cost or expected usefulness [4, 5]. AdaCache selects aligned blocks for known missing intervals [7]; WaRP plans object-store requests from intervals already known to be required [6]. Our reader instead chooses a blocking refill before the next tool read is known. It combines likely continuation with the cost of the aligned windows that the current read would cross.

We therefore ask: *does adaptive fetching reduce the time to finish the same training-execution work, excluding model inference?* This isolates the execution stack from model-serving latency without reducing the amount of tool or learner work.

2 Reader and training experiment

Choosing a refill. The reader considers aligned windows from 16 KiB to 1 MiB. Recent adjacent reads suggest that a stream will continue; jumps reduce that expectation. Completed requests update its estimate of fixed request cost and transfer rate. It selects the window with the lowest

$$\frac{\text{estimated fetch time}}{\text{bytes likely to be used}}.$$

The cost includes crossing multiple windows, not just the requested byte count. Cache capacity, replacement, workers, and in-flight limits are unchanged. Extra bytes arrive inside the blocking refill; there is no asynchronous prefetcher. The implementation and parameter definitions are in the accompanying code.

Measured tool-stage input. An independent C++ reader accesses a 512 MiB image over loopback HTTP with a 64 MiB cache. Random, sequential, and mixed traces are tested at three concurrency levels and under three controlled latency/bandwidth settings, giving 27 conditions. Each has three measured repetitions. Timing covers the entire replay process, including setup and drain. Only client-observable timings reach the controller.

All controls use the same executable. Alongside fixed256, we test other fixed sizes, choose one static setting on separate development traces, and include a simple grow/reset rule. That rule increases its window on successive missed demands and resets after a jump. Every measurement checks the returned data and common resource budgets.

Execution time excluding inference. We use each measured replay duration as the cost of one synthetic image-read job, not as a pure storage latency. A rollout consists of setup, the image-read job, additional tool computation, verification, and

Execution setting	Fixed256 (s)	Adaptive (s)	Time saved
Short compute stages	25.8	19.2	25.7%
Long tool computation	352.5	342.0	3.0%
Long learner updates	212.0	191.6	9.6%

Table 1: Inference-excluded execution time through the final learner update: geometric means across 27 read conditions. Setup, tool work, verification, teardown and learner updates remain counted. These are not end-to-end training wall-clock speedups.

teardown. Model-call placeholders take zero time and bypass the model-worker queue. We resimulate the schedule; subtracting summed inference durations from overlapping work would give the wrong clock. Actor, CPU, and tool-worker limits remain unchanged. A learner update starts only after its assigned rollouts finish; the next update's rollouts wait for that update.

Every simulated run completes the same 256 rollouts and eight learner updates. Policies share job order, service-time variation, non-inference work, and resources. We vary tool computation and learner-update cost to expose different reasons execution might wait. The clock stops after the final update, not after the last image read.

Each tool worker has an independent reader/backend budget, and each image-read job starts with an empty reader cache. This is a controlled execution-time model: it does not simulate learning quality, real repository tasks, warm sandbox reuse, or contention inside a shared 3FS service. Its clock is inference-excluded execution, not actual end-to-end training wall time. Complete parameters and individual outcomes are provided with the data.

3 What changes execution time?

Table 1 reports the execution endpoint with inference excluded. With short compute stages, adaptive fetching gives 25.7% lower completion time against fixed256. The development traces selected `fixed64`; against that static setting, execution time is 15.4% lower. Against grow/reset it is 7.0% lower in the same setting.

Stream handling provides a more serious counterexample. In the sequential condition that collapses stream IDs behind a serialized frontend, inference-excluded execution takes 3.73× fixed256 time when other stages are short. Interleaved reads no longer look sequential to the policy; the read-job records show a shift toward small refills and more requests. The frontend setting is identical in both arms. This is a simulated failure case, not an observed slowdown of DSec.

Image fetching deserves attention when it delays usable rollouts. Even without inference, CPU work and learner updates can limit the gain. Correct stream identity remains essential. Captured coding-tool traces and matched training runs are needed to assess actual time-to-quality.

Code and reproduction. Source, calibration data, the full simulation matrix, and commands for recollecting the measurements and rebuilding this paper are public at `https://github.com/majdzayyad/agent-training-image-io`. Release `v1.1.0` fixes the version used here.

References

[1] Marc Brooker, Mike Danilov, Chris Greenwood, and Phil Piwonka. On-demand container loading in AWS Lambda. arXiv:2305.13162v2, 2023. URL https://arxiv.org/abs/2305.13162.

[2] Binny S. Gill and Luis Angel D. Bathen. AMP: Adaptive multi-stream prefetching in a shared cache. In *USENIX FAST*, pages 185–198, 2007. URL https://www.usenix.org/legacy/events/fast07/tech/gill.html.

[3] Jialiang Huang, Hongxuan Tang, Jingchang Chen, et al. DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale. arXiv:2609.22978v1, 2026. URL https://arxiv.org/abs/2609.22978v1.

[4] Haneen Mohammed, Ziyun Wei, Eugene Wu, and Ravi Netravali. Continuous prefetch for interactive data applications. *Proceedings of the VLDB Endowment*, 2020. doi: 10.14778/3407790.3407826. URL https://arxiv.org/abs/2007.07858v1.

[5] R. Hugo Patterson, Garth A. Gibson, Eka Ginting, Daniel Stodolsky, and Jim Zelenka. Informed prefetching and caching. In *ACM Symposium on Operating Systems Principles (SOSP)*, 1995. doi: 10.1145/224057.224064. URL https://www.pdl.cmu.edu/PDL-FTP/TIP/SOSP15.pdf.

[6] Ruipeng Wan, Xinyu Zeng, Yiming Qiao, and Huanchen Zhang. WaRP: Workload-aware request planning for vector retrieval on object storage. arXiv:2610.29689v1, public preprint, 2026. URL https://arxiv.org/abs/2610.29689v1.

[7] Qirui Yang, Runyu Jin, Ni Fan, Devasena Inupakutika, Bridget Davis, and Ming Zhao. AdaCache: A disaggregated cache system with adaptive block size for cloud block storage. In *IEEE International Conference on Cloud Computing (CLOUD)*, 2023. doi: 10.1109/CLOUD60044.2023.00048. URL https://arxiv.org/abs/2306.17254v1.